

Labwork 3 Report: Hello, CUDA!

Lucien Elyas Bedrane

Student ID: 2640012

September 30, 2026

1 Objectives & Breakdown

In this lab, we want to implement a CPU and GPU version of a grayscale function in Python. This lab was done with Google Collab using the T4 GPU.

2 Implementations & Results

For this lab, I mainly used the implementations of grayscale used in the course as well as for the functions to copy data from the CPU to the GPU and to the GPU to the CPU.

From the hints given in the instructions of this labwork, I used the given matplotlib and numpy functions to recover the source image into a 3D array that was then flattened to a 1D array to be used by the grayscale functions later.

To test the effect of block sizes and grid sizes on how parallelization allows to increase the GPU calculations, I created a loop that times how long a grayscale iteration takes depending on the block sizes.

The following is the graph of Block size vs Time with block size ranging from 2^0 to 2^{10} :

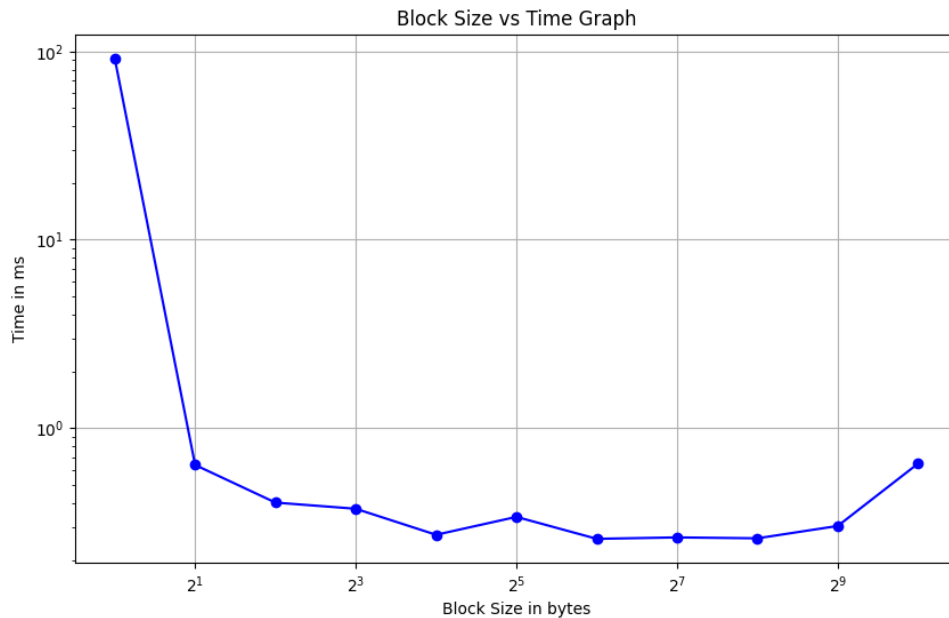


Figure 1: Block size vs Time graph.

As we can see in this graph, when the block size is 2^0 , the time is really bad as it is 10^2 slower than with bigger block size.

We can understand from this that the lower the block size is, the less we can make use of the

GPU capacity for parallelization to accelerate its calculations.

However, at a block size of 2^{10} , a warning is thrown in the execution of our program indicating that we aren't using that many blocks. In other words, we don't utilize the full capacity of the GPU making it that our execution times is slower.

Lastly, when the block size is higher than 2^{10} , the program crashes.

3 Sources & References

The following resources and documentation were used to complete this work:

- Introduction to CUDA by Tran Giang Son for M2 at ICT
- Numpy documentation for the reshape function:
<https://numpy.org/doc/stable/reference/generated/numpy.reshape.html>
- Numpy documentation for the flatten function:
<https://numpy.org/doc/stable/reference/generated/numpy.ndarray.flatten.html#numpy.ndarray.flatten>
- Matplotlib documentation for the imread function:
https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.imread.html
- Matplotlib documentation for the imsave function:
https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.imsave.html#matplotlib.pyplot.imsave